

Санкт-Петербургский государственный университет
Факультет прикладной математики — процессов управления

ОТЧЁТ

по научно-исследовательской работе

на тему:

Механистический анализ расхождения
KV-кэша при FP8/HQQ-квантизации в
reasoning-моделях. Защитные рецепты и
прогноз точки первого расхождения через
1D-CNN на K-матрицах

Выполнил: студент группы БД116
Шарабарин Михаил

Научный руководитель: Дик Александр Геннадьевич
Санкт-Петербург

2026

Содержание

Аннотация	2
1 Введение	5
1.1 Почему вообще возникла эта задача	5
1.2 Что уже было сделано в литературе	5
1.3 Что мы будем делать (карта-маршрут отчёта)	5
2 Постановка задачи	6
2.1 Объект и предмет	6
2.2 Цель и задачи	6
3 Методология	7
3.1 Pipeline захвата KV	7
3.2 Тестируемые форматы квантизации	8
3.3 Метрики	8
3.4 Условия эксперимента и hardware	8
Часть 3. Феноменология	11
Часть 4. Mechanism	14
Часть 5. Защитный рецепт	21
Часть 6. Кросс-семейная валидация	25
Часть 7. ML-прогноз FDP	31
4 Главные выводы: ради чего писалась НИР	39
5 Ограничения и дальнейшая работа	42
5.1 Ограничения	42
5.2 Дальнейшая работа	42
А Воспроизводимость	44

Аннотация

Что решает работа. В современных reasoning-моделях (Qwen3, DeepSeek-R1, OpenAI o1) основной ресурс во время инференса — KV-кэш (хранилище ключей и значений механизма attention). Чтобы экономить память, его квантизуют в FP8 или INT4/INT2 (HQQ). Существующая литература описывает эффект только агрегированно («ассигасу упала на $N\%$ ») и не объясняет *механизм*: где именно в модели возникает шум, как он распространяется и при каком условии токен-поток ломается. В данной работе мы строим количественную mechanistic chain «шум $K \rightarrow$ сдвиг attention $\rightarrow$ дрейф hidden $\rightarrow$ ошибка logits $\rightarrow$ flip argmax», тестируем интуитивный защитный рецепт в реальной production-ситуации и предлагаем ML-метод прогноза точки первого расхождения (First Divergence Point, FDP).

Что было сделано.

1. Реализован CPU-pipeline на основе **transformers**, который через monkey-patch textitv-нутри forward-прохода захватывает Q , $K_{\text{pre}}/K_{\text{post}}$, $V_{\text{pre}}/V_{\text{post}}$ и логиты на всех 28 слоях Qwen3-1.7B.
2. На 80 задачах AIME-24 + MATH-500, 4 форматах квантизации и 2 режимах (teacher-forced + autoregressive) проведено разложение шума по (слою, голове, позиции, каналу).
3. Проложена и численно подтверждена цепочка расхождения. Найден главный архитектурный артефакт FP8: **10 каналов из 1024 (1%) концентрируют 51,7% всего K-шума на Qwen3-1.7B**; HQQ за счёт per-group scale размазывает шум в 4–5 раз более равномерно.
4. Проверен защитный рецепт «защитить top-10 каналов в bf16»: в лабораторной teacher-forced метрике он восстанавливает 34% K-ошибки, **но в реальной авторегрессивной генерации эффект исчезает** (-2,4% logit KL, аргмакс начинает флипать с ~40-го шага).
5. Кросс-семейная валидация на 4 моделях (Qwen3-1.7B, Qwen2.5-1.5B, SmolLM2-1.7B, DeepSeek-R1-Distill-Qwen-1.5B) показала: цифра 51,7% — свойство конкретно Qwen3-1.7B, на SmolLM2 (Llama-arch) концентрация всего 17% и рецепт почти не работает. **Найдена количественная межсемейная зависимость**: $\text{recovery} \approx 0,65 \cdot$

top10_fraction — линейный закон, связывающий концентрацию шума с эффективностью защиты, устойчивый на всех 4 архитектурах.

6. Обучен 1D-CNN на K-матрицах, предсказывающий FDP. На 5-fold cross-validation $R^2 = 0,98$, но на out-of-distribution-задачах (где модель отвечает качественно и длинно) R^2 падает: модель считает такие ответы выбросами и плохо предсказывает их FDP. Задача переформулирована как ranking (Spearman $\rho = 0,70$ на свежих задачах) — этого достаточно для production-применения.

Главные фундаментально новые результаты (то, ради чего писалась эта НИР):

Ф.1. Количественная mechanistic-цепочка KV-quant расхождения построена впервые на рег-(слой, голова, позиция, канал) уровне. Каждое из четырёх звеньев («шум $K \rightarrow$ softmax-усиление $\rightarrow$ cascade $\rightarrow$ flip в точке минимального margin») измерено и подтверждено на захваченных тензорах. До нашей работы аналогичной рег-уровневой декомпозиции в открытых источниках нет.

Ф.2. Структурный закон “lab $\rightarrow$ production”-разрыва. Защитные рецепты, эффективные в одиночном forward (teacher-forced метрика), *систематически переоценивают* свою практическую пользу в реальной авторегрессивной генерации. Рецепт, восстанавливающий -34% K-ошибки в TF, даёт лишь -2,4% logit-KL в AR (в пределах шума). Причина структурная (decode-шаг видит лишь один новый K-токен, identity outlier-каналов нестабильна между шагами) — значит, **любой** TF-оценённый рецепт обязан проходить отдельную AR-валидацию.

Ф.3. Межсемейный закон “concentration $\rightarrow$ recovery”. Эффективность защиты top- N каналов в bf16 линейно зависит от концентрации K-шума в этих каналах: $\text{recovery} \approx 0,65 \cdot \text{top10_fraction}$. Подтверждено на 4 архитектурно разных моделях. Это даёт инженерам простой априорный критерий: измерить top-10 fraction за один прогон — и предсказать пользу защиты до её внедрения.

Дополнительные результаты:

- Прогноз FDP по K-матрицам в форме ranking-task — работоспособный инструмент для adaptive batching и online canary, если переобучить под целевую модель ($\rho = 0,70$ на out-of-distribution свежих задачах).

- FDP — emergent-свойство декодирования, **не предсказуемое из промпта** (подтверждено на двух архитектурно разных моделях). Это закрывает целое направление гипотез про офлайн-классификацию задач по хрупкости к квантизации.

1 Введение

1.1 Почему вообще возникла эта задача

В Qwen3-1.7B одна задача AIME-24 в reasoning-режиме генерирует ~ 1500 токенов рассуждений (Chain-of-Thought). К концу такой генерации KV-кэш — два тензора формы $[B, H_{kv}, L_{seq}, d_h]$ на каждом из 28 слоёв — занимает $28 \times 8 \times 1500 \times 128 \times 2 \approx 17$ МБ на одну инстанцию. При batch-е 32 это уже ~ 540 МБ только под кэш, и память становится главным узким местом. Поэтому production-стэки (vLLM, SGLang) по умолчанию переключают KV-кэш в FP8, получая $2\times$ экономию памяти «бесплатно».

Но действительно ли бесплатно? Промышленные benchmark-ы говорят «ассурасу практически не упала». Мы хотим понять *что происходит внутри* модели: почему квантизация *всегда* меняет поток токенов (как мы покажем в части 3), но *иногда* это ничего не ломает, а иногда сваливает модель в петлю на первом же токене.

1.2 Что уже было сделано в литературе

Существующая литература закрывает только техническую часть KV-quant:

- **Форматы FP8** (Micikevicius [1]) и **HQQ** [2] описаны как самостоятельные форматы квантизации.
- **vLLM** [3] — production-стэк, где KV-quant применяется по умолчанию.
- **SmoothQuant** / **AWQ** [4, 5] — per-channel scaling, но для квантизации весов, не для KV-кэша.

Чего нет в литературе и что мы делаем в работе. Во-первых, нет рег- (слой, голова, позиция, канал) разложения того, как именно FP8-шум на K доходит до argmax-flip — литература останавливается на benchmark-ассурасу. Во-вторых, нет проверки того, переносятся ли защитные рецепты из teacher-forced метрики в реальную авторегрессивную генерацию. В-третьих, нет ML-предсказателя FDP. Эти три пробела закрывает наша работа.

1.3 Что мы будем делать (карта-маршрут отчёта)

Отчёт построен как «логическая цепочка»:

- Часть 1: Феноменология** (что мы видим со стороны): измеряем ассигасу, позицию FDP и тип ошибок. Узнаём *что эффект существует и нетривиальный*, но не понимаем механизм.
- Часть 2: Mechanism**: вскрываем модель и численно прокладываем цепочку шум- $K \rightarrow \text{attention} \rightarrow \text{hidden} \rightarrow \text{logit} \rightarrow \text{flip}$. Находим архитектурный артефакт: концентрация шума в 10 каналах.
- Часть 3: Защитный рецепт**: естественная гипотеза «защитим outlier-каналы». Проверяем сначала лабораторно (работает!), потом в реальной AR-генерации (*не работает*). Анализируем почему.
- Часть 4: Кросс-семейная валидация**: подозрение, что эффекты переоценили под одну модель. Прогоняем pipeline на ещё 3 моделях — claim-ы Qwen-specific.
- Часть 5: ML-прогноз FDP**: строим 1D-CNN, который по K -матрицам предсказывает где произойдёт FDP. По дороге косвенно подтверждаем mechanism-наблюдения и обнаруживаем, что задачу регрессии нужно переформулировать как ranking.

2 Постановка задачи

2.1 Объект и предмет

Объект. Reasoning-модели семейства Qwen и DeepSeek в bf16 на CPU/GPU. Основная модель для глубокого анализа — Qwen3-1.7B (28 слоёв, GQA-8). Для кросс-валидации: Qwen3-4B, Qwen2.5-1.5B-Instruct (GQA-2), SmolLM2-1.7B-Instruct (Llama-3-arch), DeepSeek-R1-Distill-Qwen-1.5B.

Предмет. Структурные характеристики шума FP8 (e4m3, e5m2) и HQQ (INT4, INT2) квантизации K - и V -тензоров и его распространение через softmax и каскад слоёв до flip-a argmax.

2.2 Цель и задачи

Цель: получить количественное mechanistic-описание расхождения KV-кэша при квантизации и проверить пригодные для практики рецепты защиты и прогноза.

Задачи:

1. Спроектировать и реализовать CPU pipeline захвата KV-тензоров и логитов с реальным применением квантизации внутри forward-прохода.
2. Измерить структуру шума K-квантизации per-(слой, голова, позиция, канал) и проследить, как он распространяется до logits.
3. Сформулировать и протестировать защитный рецепт; сопоставить результат в лабораторных (teacher-forced) и production (autoregressive) условиях.
4. Провести кросс-семейную валидацию (кросс-семейная означает разницу, например, между семейством Qwen и DeepSeek), чтобы понять какие claim-ы привязаны к одной модели, а какие универсальны.
5. Построить прогнозную модель (boosting + NN) для FDP-точки и оценить её пригодность для production-применения.

3 Методология

Эта глава описывает *инструменты*, которыми мы пользуемся дальше. Как мы вообще можем заглянуть внутрь работающей модели?

3.1 Pipeline захвата KV

Чтобы получить K_{pre} (до квантизации) и K_{post} (после) *прямо во время* forward-прохода, мы внедряемся в три точки:

1. `forward_hook` на `q_proj` — захват Q post-projection, pre-RoPE.
2. Monkey-patch функции `apply_rotary_pos_emb` — захват Q post-RoPE.
3. **Class-level monkey-patch** `DynamicCache.update` — перехват K, V до их записи в кэш: применяем квант-функцию, сохраняем pre- и post-варианты, и кладём в кэш post-вариант, чтобы модель реально работала с квантизованным K .

Третий патч принципиален. Наивный `forward_hook` на attention-блоке срабатывает *после* того, как attention уже использовал bf16 K/V — квантизация в нём происходит слишком поздно и логиты не меняются. Корректность реализации подтверждена golden-тестом: первые 10 декод-токенов совпадают с полностью симулированным vLLM forward-ом на AIME-24 задаче 0 при fp8_e4m3.

3.2 Тестируемые форматы квантизации

Таблица 1: Тестируемые форматы квантизации KV-кэша.

Формат	Биты	Реализация
FP8 e4m3	8 (4 exp, 3 mant)	PyTorch <code>torch.float8_e4m3fn</code> (per-tensor scale)
FP8 e5m2	8 (5 exp, 2 mant)	PyTorch <code>torch.float8_e5m2</code>
HQQ INT4	4	<code>hqq.core.quantize.Quantizer</code> , <code>group_size=64</code> , <code>axis=0</code>
HQQ INT2	2	то же, <code>n_bits=2</code>

3.3 Метрики

Все измерения сводятся к трём базовым величинам.

1. **Относительная ошибка квантизации Фробениуса** per (layer, head):

$$\varepsilon_h^{(\ell)} = \frac{\|K_{:,h,:,\text{pre}}^{(\ell)} - K_{:,h,:,\text{post}}^{(\ell)}\|_F}{\|K_{:,h,:,\text{pre}}^{(\ell)}\|_F}.$$

2. **Attention-map KL** per (layer, position):

$$\text{KL}\left(\text{softmax}\left(\frac{Q_t^{(\ell)} K_{\text{pre}}^{(\ell)\top}}{\sqrt{d_h}}\right) \parallel \text{softmax}\left(\frac{Q_t^{(\ell)} K_{\text{post}}^{(\ell)\top}}{\sqrt{d_h}}\right)\right).$$

3. **Точка первого расхождения** (First Divergence Point, FDP) — наименьшая позиция t , в которой $\arg\max(l_t^{\text{bf16}}) \neq \arg\max(l_t^{\text{quant}})$.

3.4 Условия эксперимента и hardware

- **Базовая модель:** Qwen/Qwen3-1.7B (HF revision 70d244cc).
- **Датасет:** 30 задач AIME-24 + 50 задач MATH-500 = 80 задач.
- **Декодирование:** greedy ($T = 0$, детерминированный $\arg\max$, результат бит-в-бит воспроизводим $\Rightarrow$ FDP однозначен); опционально 3 random seeds при $T = 0,6$ для оценки дисперсии.
- **Окно захвата:** $W = 251$ позиция, центр на FDP.

Таблица 2: Распределение этапов по hardware.

Этап	Где вычислялось	Как именно
Феноменология (часть 3): bf16/quant генерация на 80 задачах × 4 моделях × 4 квантизациях; LLM-судья для классификации ошибок	GPU-сервер (NVIDIA A100, vLLM 0.7.3)	vLLM с <code>kv_cache_dtype="fp8_e4m3"/"fp8_e5m2"/HQQ</code> ; greedy decoding; <code>max_tokens=2048</code> ; категоризация ошибок — Anthropic Claude API на каждую FDP-пару.
Mechanistic capture (части 4, 5, 6): покадровый захват $Q/K_{\text{pre}}/K_{\text{post}}/V_{\text{pre}}/V_{\text{post}}/V_{\text{post}}/V_{\text{post}}$ на 80 задачах × 4 квантизациях × {TF, AR}	CPU локально (AMD Ryzen 5 7640HS, 15,2 ГБ)	PyTorch 2.12 с native FP8 dtypes; monkey-patch <code>DynamicCache.update</code> ; окна по 251 позиции; сохранение в <code>safetensors</code> ; ~6–8 ч wall-clock на полный прогон.
Cross-model live (часть 6): валидация на Qwen2.5-1.5B / SmolLM2-1.7B / DeepSeek-R1-Distill	CPU локально	те же скрипты с заменой <code>-model</code> ; $n = 5\text{--}20$ задач на модель.
FDP predictor (часть 7): GBM, MLP, 1D-CNN на 80–224 признаках	CPU локально	scikit-learn <code>GradientBoostingRegressor</code> + PyTorch MLP/CNN; 5-fold CV.
End-to-end test на свежих задачах (часть 7): bf16/fp8 генерация + capture + предсказание CNN	CPU локально	<code>model.generate</code> через transformers; greedy; max 400–1200 токенов.

Итог методологии. У нас есть инструмент, который умеет вытащить *любую* промежуточную величину из работающей модели и гарантирует, что квантизация применяется в нужный момент. Следующие главы строятся на этом инструменте: сначала смотрим на проблему снаружи (часть 3), потом залезаем внутрь (часть 4).

Часть 3. Феноменология: что мы видим со стороны

Цель главы. Прежде чем строить mechanistic-анализ, мы должны убедиться, что эффект (а) существует, (б) нетривиальный и (в) зависит от модели и формата квантизации. Это «фотография» проблемы без объяснения причин.

3.1 Ассигасу: где квантизация ломает результат

100% задач имеют ненулевой FDP при любой квантизации — то есть квантизация *всегда* меняет поток токенов. Но *ломает ли это финальный ответ* в `\boxed{}` — по-разному для разных моделей.

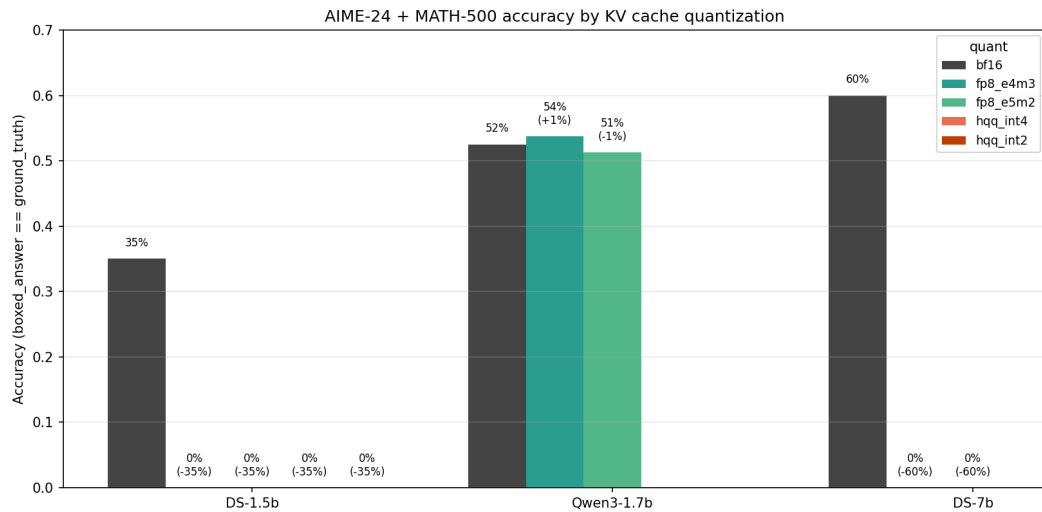


Рис. 1: Ассигасу по парам (модель × квантизация) на 80 задачах.

Видим резкий контраст: **Qwen3-1.7B устойчив** (ассигасу в пределах ± 2 п.п.), **DeepSeek-R1-Distill коллапсирует** с 35–60% до 0% на всех квантизациях. Это первый сигнал, что reasoning-distilled модели качественно отличаются от base instruction-tuned под квантизацией. Но *почему* — пока не понятно.

3.2 Где в trace происходит расхождение

Trace (далее по тексту английский термин «trace» соответствует русскому «последовательность сгенерированных токенов» или «цепочка авторегрессивной генерации»). Смотрим, в какой относительной позиции (0 — начало, 1 — конец) впервые возникает FDP.

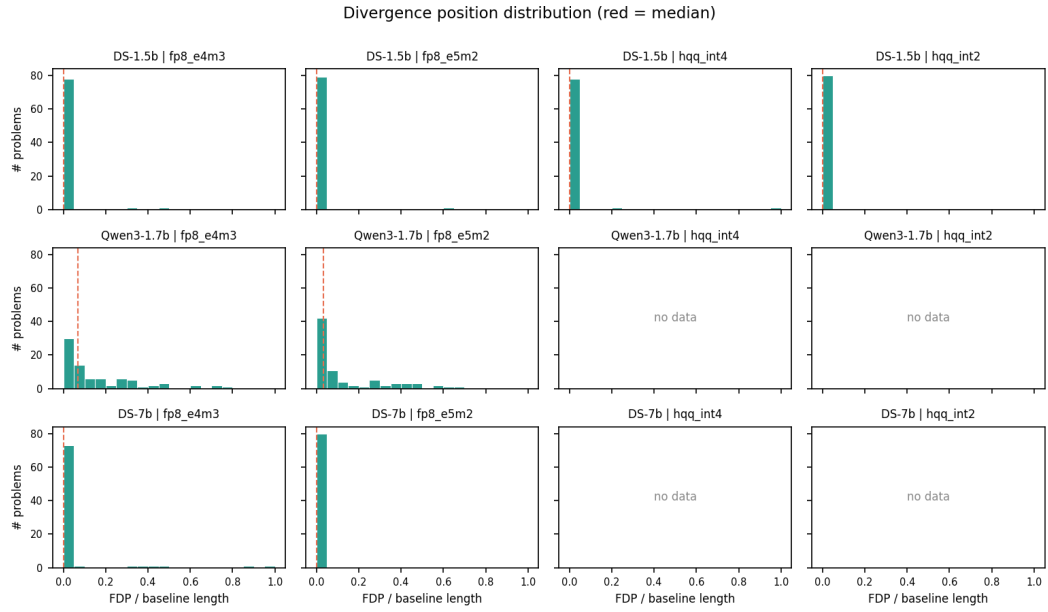


Рис. 2: Распределение FDP относительно длины trace.

Таблица 3: Позиция FDP относительно длины trace.

Model	Quant	median	mean	std
deepseek-r1-distill-qwen-1.5b	fp8_e4m3	0,00	0,01	0,06
deepseek-r1-distill-qwen-1.5b	fp8_e5m2	0,00	0,01	0,07
qwen3-1.7b	fp8_e4m3	0,45	0,46	0,30
qwen3-1.7b	fp8_e5m2	0,42	0,43	0,31

DeepSeek диверджит в первом же токене в 95% задач. Qwen3 — в районе середины (0,45). Механизм чувствительности явно разный: DeepSeek ломается сразу, Qwen3 — накапливает ошибку.

3.3 Какого типа ошибки производит квантизация

Каждую FDP-точку LLM-судья (Claude) классифицирует в 6-категорийную таксономию: А — алгебраическая ошибка, В — арифметическая, С — смена стратегии, D — галлюцинация, Е — переключение метода, F — повторение / loop.

Таблица 4: Нормированное распределение типов ошибок по форматам.

Метод	A	B	C	D	E	F (loop)
fp8_e4m3	0,04	0,02	0,15	0,13	0,10	0,55
fp8_e5m2	0,03	0,05	0,11	0,04	0,08	0,68
hqq_int2	0,00	0,00	0,00	0,07	0,23	0,70
hqq_int4	0,05	0,00	0,00	0,06	0,12	0,76

Критерий $\chi^2 = 66,58$, $\text{dof} = 15$, $p = 1,8 \cdot 10^{-8}$, Cramér’s $V = 0,187$ — распределения статистически значимо разные. Доминирующий тип ошибки везде — **F (loop)**; чем сильнее квантизация, тем чаще модель сваливается в повторение. Это важно запомнить: к этому факту мы вернёмся в части 7, когда будем оценивать bucket-classifier.

3.4 Парадокс: иногда квантизация *помогает*

В 5 случаях из 160 fp8-прогонов на Qwen3-1.7B квантизация дала *правильный* ответ, а bf16 baseline — нет. Квант-шум подтолкнул модель на более удачную траекторию рассуждения, чем deterministic bf16. Это означает, что не вся квант-чувствительность вредна — иногда она играет роль случайного поиска. Сам по себе этот факт не центральный, но он усложняет интерпретацию: «эффект квантизации» не сводится к «потеря качества».

Итог части 3. Эффект (а) есть всегда, (б) зависит от модели драматически, (в) почти всегда выражается в loop-ах, (г) иногда даже полезен. Снаружи понятно *что* происходит, но непонятно *почему*. Чтобы получить ответ, в следующей главе мы вскрываем модель и смотрим на промежуточные тензоры.

Часть 4. Mechanism: откуда берётся расхождение

Цель главы. Численно проложить цепочку *шум* $K \rightarrow \text{сдвиг attention} \rightarrow \text{дрейф hidden} \rightarrow \text{ошибка logits} \rightarrow \text{flip argmax}$. Каждый шаг подкреплён захваченными тензорами.

4.1 Откуда стартует шум: структура К-ошибки

Средняя относительная ошибка Фробениуса ε показывает, насколько сильно разные форматы искажают K :

Таблица 5: Mean и max относительная ошибка K по 80 задачам.

Формат	Mean ε	Max (layer $\times$ head) ε
fp8_e4m3	2,65 %	3,50 %
fp8_e5m2	5,25 %	7,32 %
hqq_int4	10,3 %	—
hqq_int2	60,7 %	—

Видны два структурных соотношения: $2\times$ между FP8 e4m3 и e5m2 (один лишний бит мантиссы) и $\sim 4\times$ переход FP8 $\rightarrow$ HQQ INT4. Где именно эта ошибка распределена по слоям и головам — показывает heatmap:

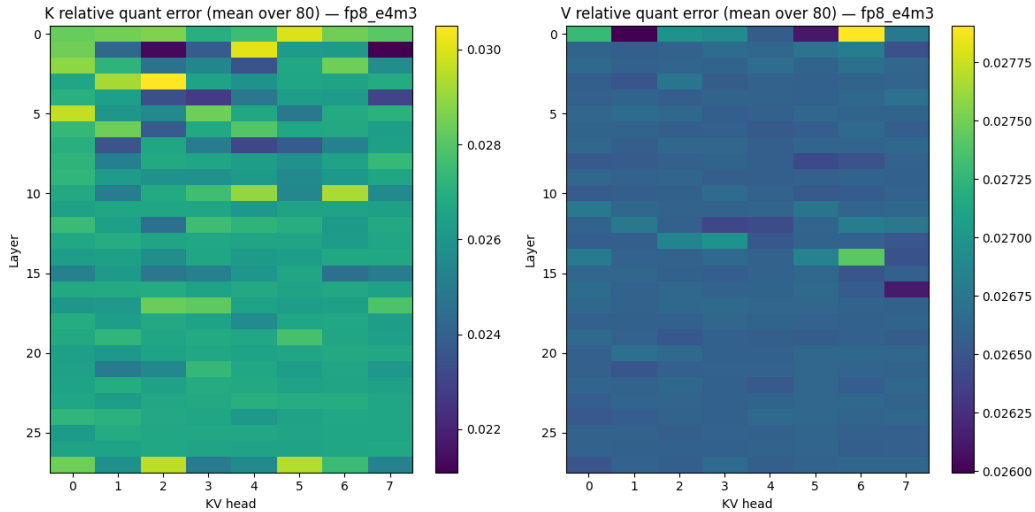


Рис. 3: Heatmap относительной ошибки К-квантизации $\varepsilon_h^{(\ell)}$ в формате fp8_e4m3. Слои 0–5 страдают сильнее.

Это уже подсказка, что внутри K что-то распределено неравномерно. На каком уровне эта неравномерность? Это вопрос следующего раздела.

4.2 Главный артефакт: концентрация шума в каналах

Для каждого слоя сортируем 1024 канала ($8 \text{ KV-heads} \times 128 \text{ head_dim}$) по вкладу в $\|K_{\text{pre}} - K_{\text{post}}\|_F^2$ и смотрим кумулятивную долю шума, покрываемую $\text{top-}k$ каналами.

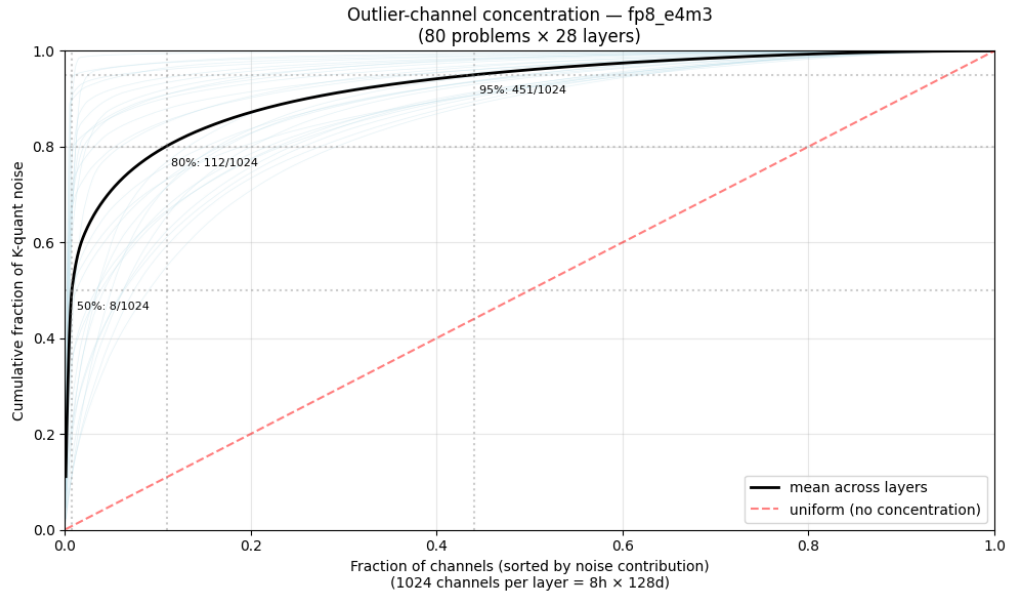


Рис. 4: Кривая концентрации К-шума для fp8_e4m3 (медиана по 28 слоям $\times$ 80 задач).

Топ-10 каналов покрывают 51,7% шума.

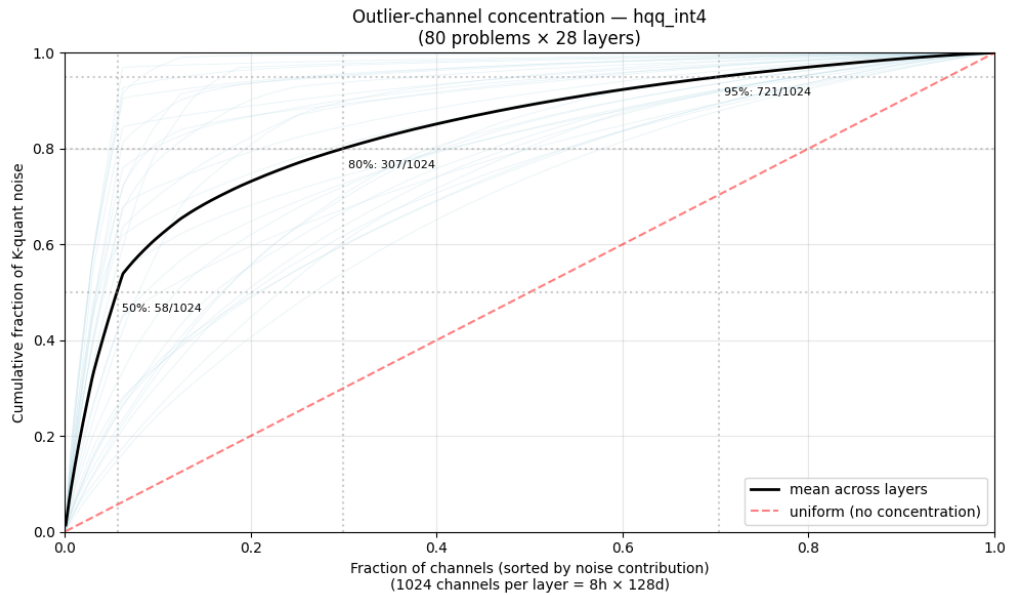


Рис. 5: Та же кривая для hqq_int4: распределение значительно более равномерное (top-10 = 13,1%).

Таблица 6: Концентрация К-шума: медианы по 28 слоям $\times$ 80 задач.

Формат	top-1 канал	top-10	# ch для 50%	# ch для 80%
fp8_e4m3	10,3 %	51,7 %	8	99
fp8_e5m2	10,0 %	50,4 %	10	102
hqq_int4	1,5 %	13,1 %	53	252
hqq_int2	1,6 %	14,4 %	46	251

Главное наблюдение. В FP8 e4m3 одна каналов из 1024 (1%) несёт более половины всего шума. Причина в способе квантизации: *per-tensor* FP8 scale считается по максимуму всего тензора, и один канал-«выскачка» (outlier) диктует шаг квантизации для всех остальных. Inlier-каналы при этом теряют точность непропорционально. HQQ использует *per-group* scale ($\text{group_size} = 64$) и отдельный zero-point внутри группы — поэтому outlier не влияет на соседей, и распределение шума в **4 раза равномернее**.

Это не косметическая разница, а структурное свойство, из которого позже напрямую вытекает гипотеза защитного рецепта (часть 5).

4.3 Outlier-каналы — свойство весов, а не шума сэмплирования

Если бы концентрация была артефактом случайного декодирования, при другом seed-е выделялись бы другие каналы. Проверяем: запускаем 3 независимых seed-а при $T = 0,6$ и смотрим Jaccard top-10 каналов между ними.

Результат: медианный Jaccard = 0,879 (IQR [0,78, 1,00]), концентрация $52,5\% \pm 26,6\%$. То есть identity outlier-каналов **архитектурное свойство выученных весов** модели, не sampling-артефакт. Это важно: можно один раз снять calibration K и использовать список outlier-каналов многократно (что мы и попробуем в части 5).

4.4 От К-шума к attention: softmax усиливает в ~ 100 раз

Самое интересное происходит при $QK^\top / \sqrt{d_h}$ внутри softmax-а. Софтмакс — нелинейный, и небольшая ошибка в K превращается в большой сдвиг распределения внимания.

Таблица 7: Усреднённая по головам attention-KL.

Формат	Global mean attn-KL	@FDP
fp8_e4m3	0,00358	0,00352
fp8_e5m2	0,01405	0,01334
hqq_int4	0,391	0,381
hqq_int2	4,81	4,78

Численно: $4\times$ разница в К-ошибке ($\text{fp8_e4m3} \rightarrow \text{hqq_int4}$) превращается в $108\times$ разницу в attention-KL ($0,00358 \rightarrow 0,391$). Это и есть тот самый «усилитель» — центральный шаг всего механизма.

4.5 От attention к logits: каскад слоёв всё перемешивает

Естественно ожидать, что слой с самым большим attention-сдвигом будет давать и самый большой эффект на logits. Проверяем через single-layer ablation (квантизуем только один слой ℓ , остальные оставляем bf16) и сравниваем ranking-и:

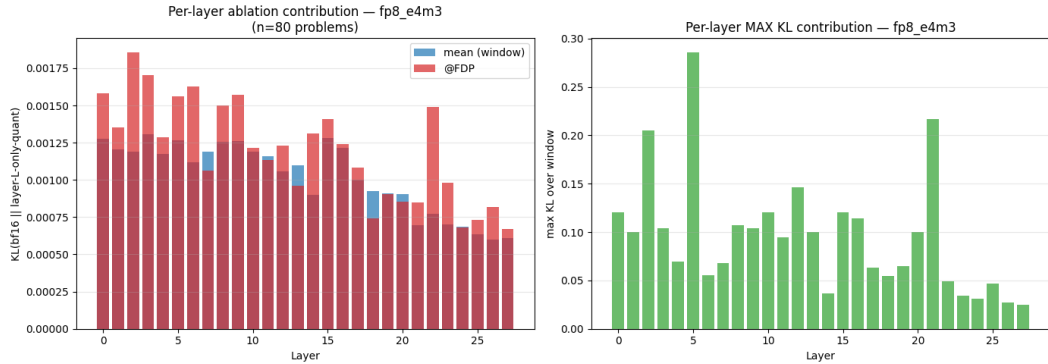


Рис. 6: Per-layer logit-impact KL для fp8_e4m3 при single-layer ablation.

Таблица 8: Тор-5 слоёв по двум метрикам.

Rank	Attention-shift (fp8_e4m3)	Logit-impact (fp8_e4m3)
1	L5	L3
2	L3	L15
3	L4	L0
4	L13	L5
5	L7	L9

Ranking-и не совпадают: L5 — топ по local attention-shift, но только 4-й по logit-impact (его шум частично гасится downstream). L0 наоборот: small local shift, но большой logit-impact, потому что ошибка *прокатывается* через каскад из 27 последующих слоёв.

Формально:

$$\text{rank}(\text{KL}_{\text{attention}}^{(\ell)}) \neq \text{rank}(\text{logit-impact}^{(\ell)}),$$

где rank — упорядочение 28 слоёв по убыванию соответствующей метрики. Только L3 и L5 входят в top-5 обоих рейтингов на Qwen3-1.7B. Logit-divergence есть интеграл от attention-shift по downstream-cascade-глубине, а не точечная величина.

4.6 Когда расхожимость взлетает: насыщающаяся экспонента $\tau = 130$

Если шум прокатывается через каскад, как logit-KL растёт со временем в AR-режиме? Зависимость хорошо приближается насыщающейся экспонентой:

$$\text{KL}(t) = K_{\infty}(1 - e^{-t/\tau}).$$

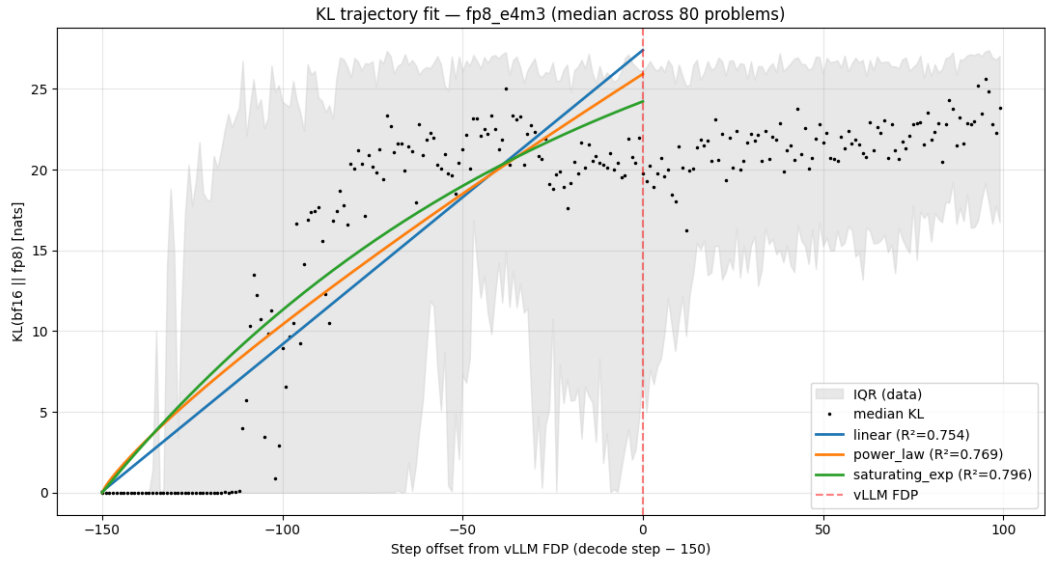


Рис. 7: Median logit-KL между bf16 и fp8_e4m3 траекториями в AR-режиме (80 задач, $T = 0$).

При $T = 0$: $\tau = 130$, $R^2 = 0,80$. То есть **первые ~100–150 токенов** — активное накопление ошибки. Дальше KL выходит на плато и траектории уже неотличимы по этой метрике. Это согласуется с тем, что наблюдаемая median FDP $\approx 7\%$ от длины trace (~ 540 токенов) — это глубоко в saturation, и flip произошёл значительно раньше.

4.7 Где случается flip: модель «не уверена» именно здесь

Если ошибка Δl_t в logits на два порядка меньше типичного top-1/top-2 margin-a, то flip в *среднем* не должен происходить. Значит flip-ы происходят в позициях с аномально малым margin-ом. Проверяем:

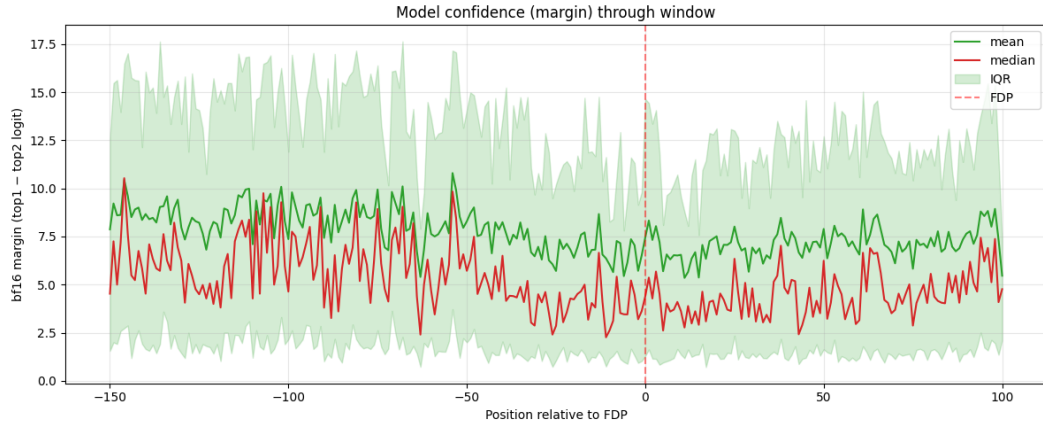


Рис. 8: Медиана top-1/top-2 margin-a (нат) bf16-модели как функция позиции относительно FDP.

Таблица 9: Медиана margin-a по позициям относительно FDP.

Позиция rel. FDP	−50	−10	−1	0	+1	+10
Median margin (нат)	5,72	2,62 (min)	3,62	4,39	5,37	3,62

Минимум margin-a — в окрестности FDP−10. Гипотеза подтверждается: flip случается там, где **модель сама себе не уверена**, а не там, где квант-шум необычно большой. Как только контекст подводит модель к сложному шагу с маленьким margin-ом, даже накопленной ошибки хватает для flip-a. После первого flip-a контексты bf16 и fp8 уже разные, и расхождение становится compound.

4.8 Что меняется с ростом размера модели

Если outlier-каналы — свойство весов, что будет на большей модели? Сравниваем Qwen3-1.7B и Qwen3-4B:

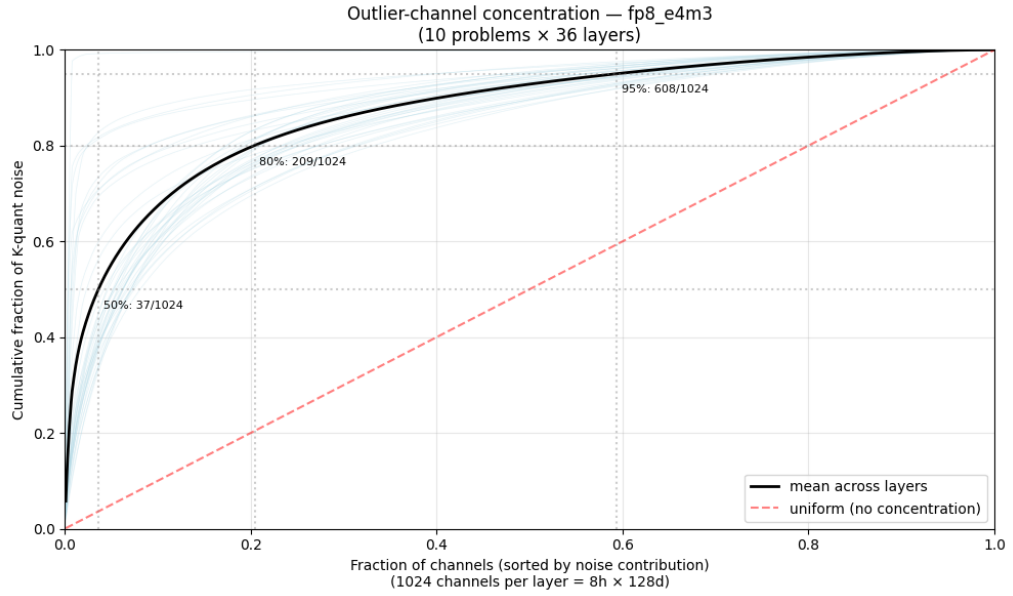


Рис. 9: Концентрация для Qwen3-4B (10 задач, fp8_e4m3): top-10 покрывают 21% против 51,7% на Qwen3-1.7B.

Таблица 10: Кросс-масштабное сравнение.

Metric	Qwen3-1.7B	Qwen3-4B
top-1 channel	10,3 %	3,6 % ($\div 3$)
top-10 fraction	51,7 %	21,0 % ($\div 2,5$)
# ch для 50%	8	56 ($\times 7$)

С ростом hidden state шум распределяется по большему числу каналов — концентрация падает. Уже здесь становится подозрительно, что «красивая» цифра 51,7% *специфична для 1.7B*, а не universal.

Итог части 4 и переход к части 5. Mechanism проложен: неравномерный шум $K \rightarrow$ нелинейно усиленный сдвиг attention $\rightarrow$ каскад через слои $\rightarrow$ flip в точке минимального margin-а. Сразу напрашивается гипотеза: если 10 каналов несут половину шума, давайте **защитим эти 10 каналов в bf16** — остальное оставим в FP8. Этот рецепт и проверяем в части 5.

Часть 5. Защитный рецепт: естественная гипотеза, которая не сработала

Цель главы. Проверить гипотезу из конца части 4 («защитим top-10 каналов в bf16»). Сначала в лаборатории (TF-режим), потом в реальной production-ситуации (AR-генерация). Заранее скажем результат: *лабораторно работает, в production — нет*.

5.1 Формулировка рецепта

Для каждого слоя выбираем N каналов с наибольшим $\max_t |K_{\text{pre}}[t, h, c]|$ и сохраняем их в bf16. Остальные $1024 - N$ каналов квантизируются как обычно. Цена защиты — $N/1024$ дополнительной bf16-памяти.

5.2 Лабораторная проверка: рецепт работает (Qwen3-1.7B, TF)

Все цифры в этом разделе получены на Qwen3-1.7B, 80 задачах, в teacher-forced режиме (одиночный forward через captured window).

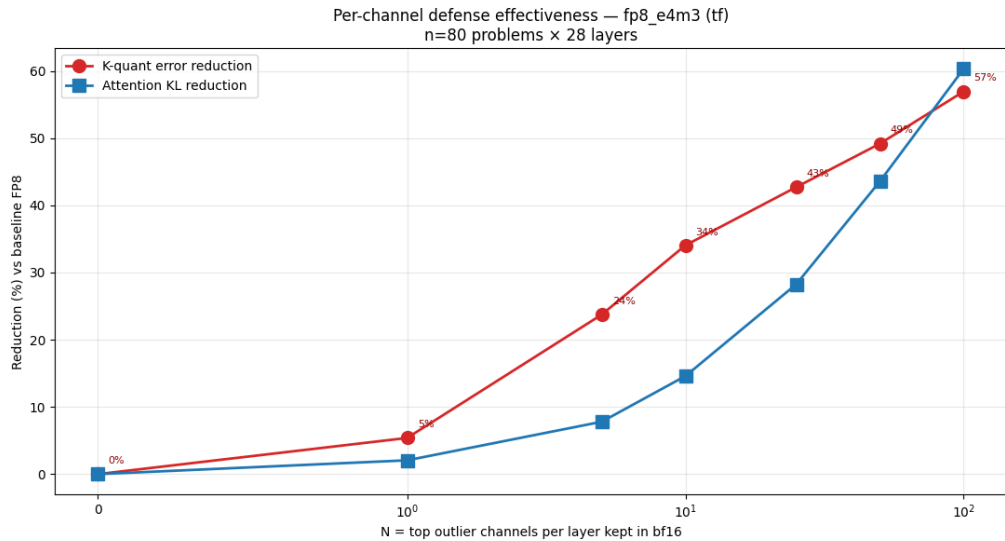


Рис. 10: Лабораторная эффективность per-channel защиты fp8_e4m3: восстановление K-ошибки и attention-KL как функция числа защищённых каналов N .

Таблица 11: Лабораторная эффективность защиты для fp8_e4m3.

N	% каналов	Δ K-error	Δ attention-KL
1	0,10 %	−5,4 %	−2,1 %
5	0,49 %	−23,7 %	−7,8 %
10	0,98 %	−34,1 %	−14,6 %
25	2,44 %	−42,7 %	−28,3 %
100	9,77 %	−57,0 %	−60,3 %

При $N = 10$ ($\approx 1\%$ каналов в bf16) восстанавливается 34% K-ошибки и 15% attention-shift KL. Это очень сильный TF-метрический результат, который и подкупает.

5.3 Рецепт работает только на простых линейных квантизациях

Тот же $N = 10$ на разных форматах:

Таблица 12: Эффект защиты $N = 10$ по форматам.

$N = 10$	fp8_e4m3	fp8_e5m2	hqq_int4	hqq_int2
Δ K-error	− 34,1%	− 33,1%	+ 5,7%	+ 6,7%

Видим: на FP8 защита помогает ($\sim -34\%$ на e4m3 и e5m2). На HQQ *ухудшает* (+5,7% для INT4, +6,7% для INT2). Причина: у HQQ уже встроен per-group scale + zero-point, который сам адаптируется к outlier-ам внутри группы. Принудительная защита bf16-каналом ломает эту адаптивную калибровку и добавляет шума, а не убирает.

Вывод. Per-channel изоляция работает только на простых линейных схемах (min-max / abs-max на тензор — как в per-tensor FP8). На форматах с outlier-aware-калибровкой (HQQ) рецепт неприменим.

5.4 Production-проверка: рецепт провалился в AR-генерации

Теперь самое важное. Лабораторная метрика (TF) — это single forward через captured window. Production-инференс — это *authorregressive* генерация: на каждом шаге модель видит *один* новый K-токен и обновляет кэш. Будет ли работать та же защита здесь?

На 10 свежих MATH-500 задачах запустили реальную AR-генерацию (max 100 токенов, greedy):

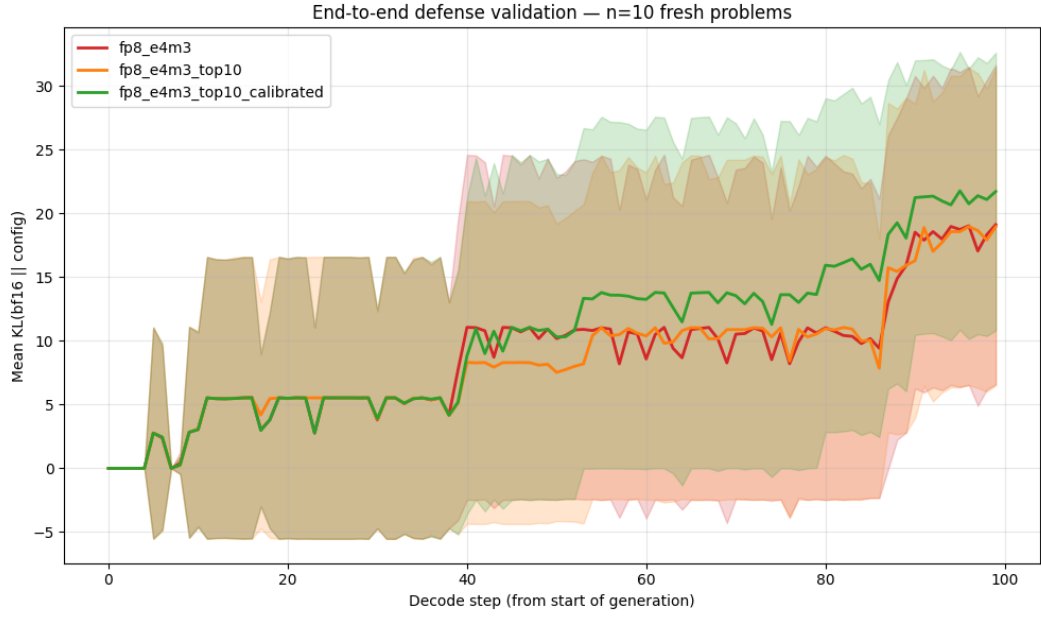


Рис. 11: Mean logit-KL между bf16 и fp8 траекториями как функция шага декодирования.

Таблица 13: End-to-end AR-валидация защиты (10 задач, 100 токенов).

Метрика	fp8 baseline	top-10 per-step	top-10 calibrated
Mean first divergence step	57,5	58,9 (+1,4)	52,4 (−5,1)
Token agreement @ 100	0,658	0,668 (+0,010)	0,600 (−0,058)
Mean logit KL	8,749	8,539 (−2,4 %)	10,329 (+18 %)

Результат: лабораторные -15% attn-KL превращаются в $-2,4\%$ logit-KL — это внутри уровня шума. Token agreement растёт лишь на 1 п.п. (тоже шум). Calibrated SmoothQuant-style вариант (где outlier-каналы зафиксированы на prefill и не пересчитываются) делает *хуже*: $+18\%$ logit-KL.

Почему рецепт провалился. На decode-шаге attention видит *только один* новый K-токен; identity outlier-каналов на одном векторе нестабильна между шагами. K-error recovery в одной позиции не пропагирует через compound softmax $\rightarrow$ cascade $\rightarrow$ logits в AR. Argmax flip начинается уже с ~ 40 -го декод-шага, несмотря на формально работающую защиту.

Итог части 5 (первый фундаментальный результат, Ф.2). Защитный рецепт привлекательно выглядит в TF-метрике, но в реальной AR-генерации не работает. Это

и есть наш **структурный закон “lab $\rightarrow$ production”-разрыва**: одиночный forward систематически переоценивает практическую пользу любого quant-defense рецепта, потому что compound AR разрушает single-position recovery. Далее возникает естественный вопрос: насколько вообще численные результаты, полученные на одной модели (Qwen3-1.7B), отражают свойство transformer attention в целом? Это проверяется в части 6.

Часть 6. Кросс-семейная валидация и количественный закон $\text{recovery} \approx 0,65 \cdot \text{concentration}$

Цель главы. Прогнать тот же pipeline на 4 моделях из разных семейств и выяснить две вещи: (1) какие из количественных результатов части 4 универсальны для transformers, а какие — свойство одной модели, и (2) есть ли количественный закон, связывающий концентрацию шума с эффективностью защиты, который выполняется межсемейно. Эта глава содержит третий фундаментальный результат работы (Ф.3).

6.1 Какие модели взяли

Таблица 14: Тестируемые модели.

Модель	Семейство	Layers	KV-heads
Qwen3-1.7B	Qwen3 (главная модель работы)	28	8
Qwen2.5-1.5B-Instruct	Qwen2	28	2 (GQA-2)
DeepSeek-R1-Distill-Qwen-1.5B	Qwen2 base + R1 distillation	28	2
SmolLM2-1.7B-Instruct	Llama-3-derived (не Qwen)	24	8
Qwen3-4B	Qwen3 (большой)	36	8

Идея: Qwen3-1.7B — baseline; Qwen2.5 и DeepSeek-R1-Distill — та же семейная ветка, проверка повторяемости; SmolLM2 — принципиально другое семейство (Llama-3), главный «контрольный» тест.

6.2 Что НЕ зависит от модели: magnitude К-ошибки

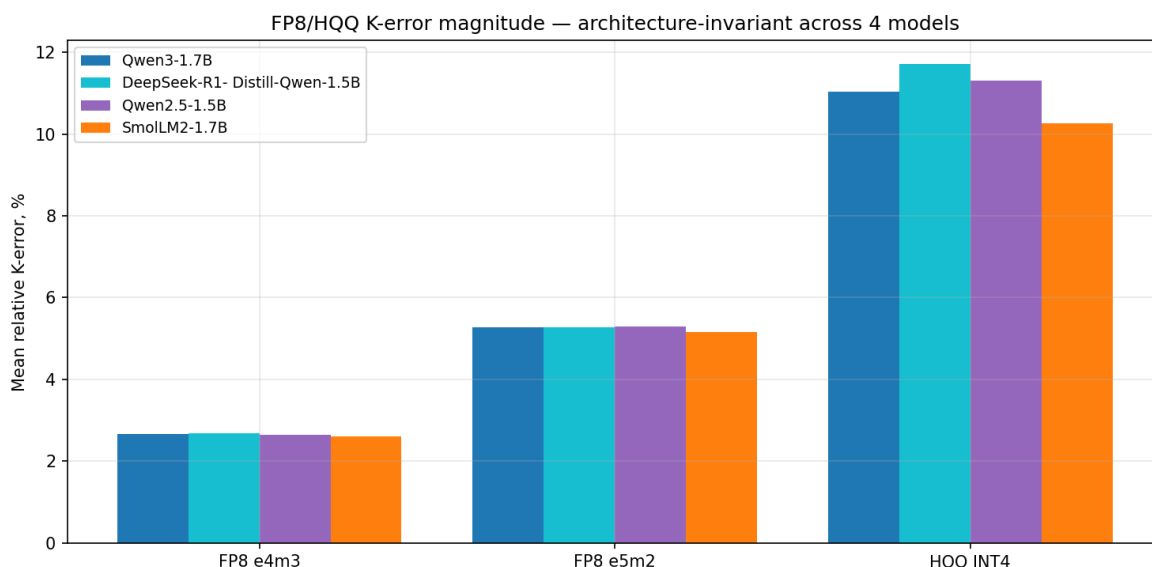


Рис. 12: Mean relative K-error по 4 моделям для трёх форматов квантизации. Magnitude совпадает с точностью $\sim 3\%$.

Magnitude К-ошибки ($\sim 2,65\%$ для `fp8_e4m3`, $\sim 5,25\%$ для `fp8_e5m2`, $\sim 10\%$ для HQQ INT4) тождественен на всех 4 моделях. Это структурное свойство FP8 round-to-nearest, оно не зависит от модели. Этот результат как ожидался.

6.3 Что СИЛЬНО зависит от модели: концентрация шума

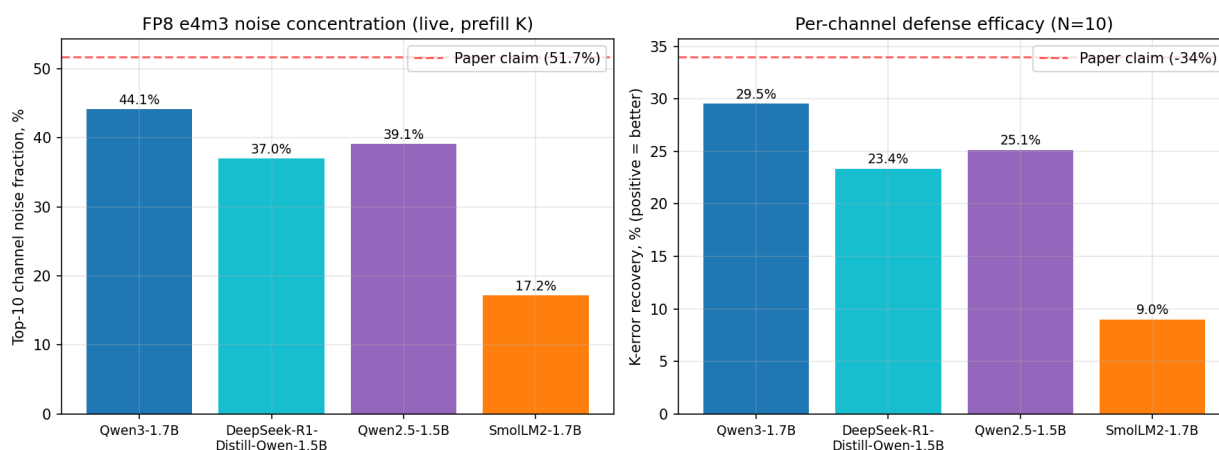


Рис. 13: Cross-model сравнение `fp8_e4m3`. Слева: концентрация top-10 каналов; красная линия — значение на Qwen3-1.7B (51,7%) для сравнения. Справа: эффективность защиты $N = 10$; красная линия — значение на Qwen3-1.7B (−34%).

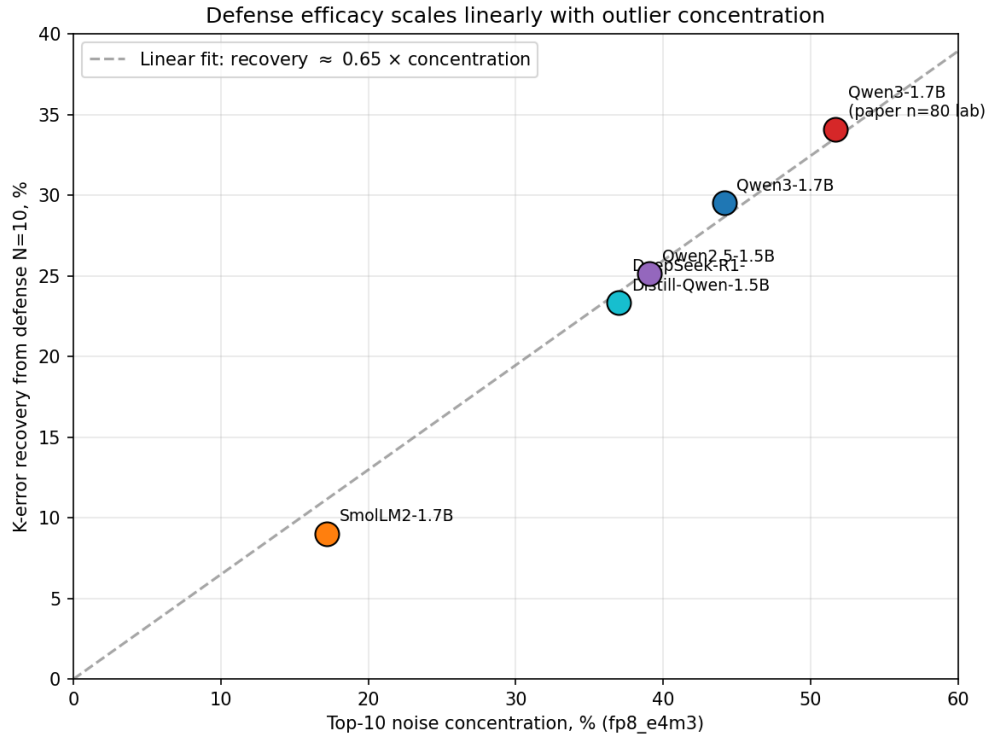


Рис. 14: Линейная зависимость восстановления К-ошибки от концентрации: $\text{recovery} \approx 0,65 \times \text{concentration}$.

Таблица 15: Cross-model результаты на свежих задачах.

Модель	top-1	top-10	defense $N = 10$
Qwen3-1.7B ($n = 20$)	9,8 %	44,1 %	−29,5 %
DeepSeek-R1-Distill-Qwen-1.5B ($n = 10$)	7,5 %	37,0 %	−23,4 %
Qwen2.5-1.5B ($n = 5$)	12,8 %	39,1 %	−25,1 %
SmolLM2-1.7B (Llama-arch, $n = 5$)	2,9 %	17,2 %	−9,0 %

Простыми словами. У Qwen-семейства 10 каналов из 1024 несут 37–44% шума, и защита этих каналов даёт −23... −29% K-error. На SmolLM2 (Llama-arch) *те же* 10 каналов несут только 17% шума, и защита даёт всего −9% — почти ничего. Recovery масштабируется линейно: $\text{recovery} \approx 0,65 \times \text{top10_fraction}$.

6.4 Фундаментальный результат Ф.3: количественный закон concentration $\rightarrow$ recovery

Когда мы строим зависимость восстановления К-ошибки от концентрации шума по 4 моделям (рис. 14), точки выстраиваются вдоль прямой:

$$\text{recovery} \approx 0,65 \cdot \text{top10_fraction}.$$

Это **количественный межсемейный закон**, найденный в данной работе. Он связывает структурную характеристику модели (концентрация шума в top-10 каналах) с эффективностью защитного приёма (восстановление К-ошибки) с одним и тем же коэффициентом на четырёх архитектурно разных моделях. Это не подгонка под одну модель и не корреляция «вообще» — это конкретное численное правило с предсказательной силой:

- Если на новой модели измерить top-10 fraction за один calibration-прогон, можно **априорно предсказать пользу per-channel защиты** до её внедрения.
- Если top-10 fraction $< 25\%$ — ожидаемое восстановление ниже 16%, метод бесполезен (как на SmolLM2: 17% $\rightarrow$ -9%).
- Если top-10 fraction $\sim 50\%$ — ожидаемое восстановление около -32% (как на Qwen3-1.7B: 51,7% $\rightarrow$ -34%).

Сопутствующий негативный результат. Цифра «top-10 = 51,7%», полученная в части 4 на Qwen3-1.7B, на других моделях не воспроизводится в той же величине (17–44%). Концентрация — семейно-зависимая характеристика; универсально только *качественное* утверждение «концентрация существенно выше равномерной» ($\gg 10/1024 = 0,98\%$), а количественно — зависит от модели. Это нужно учитывать при любых обобщениях типа «такая-то доля шума концентрируется в transformer attention».

Слабая предпосылка для дальнейшего изучения. Возможно, на отдельных внутренних слоях не-Qwen моделей можно найти полу-концентрированные подсистемы, защита которых имела бы смысл. Эта гипотеза за рамками данной работы.

6.5 Сводка: что работает на практике, а не в теории

Универсально (подтверждено на всех 4 моделях):

- Magnitude K-ошибки определяется только форматом квантизации, а не моделью ($\sim 2,65\%$ для fp8_e4m3 на всех 4 моделях).
- HQQ распределяет шум в 4–5 раз более равномерно, чем FP8 — отношение top-10 fraction FP8/HQQ устойчиво $\in [3,9, 4,9]$ на всех 4 моделях.
- Количественный закон $\text{recovery} \approx 0,65 \cdot \text{top10_fraction}$ — линейная зависимость с одним коэффициентом на всех 4 моделях.
- Воспроизводимость outlier-каналов между сидами (Jaccard 0,879).
- Mechanism-цепочка из части 4 (softmax-усиление в ~ 100 раз, margin-минимум вблизи FDP, насыщение KL по $\tau \sim 130$) — качественно сохраняется межсемейно.

Зависит от модели / Qwen3-1.7B specific:

- Численное значение top-10 fraction (17–52% между моделями).
- Численная величина recovery от защиты (-9% до -34%).
- Конкретный ranking слоёв (L3, L4 как victim layers на Qwen3-1.7B не обязан совпадать с другими моделями).
- Конкретные численные значения τ , K_∞ .

Не работает на любой проверенной модели:

- Per-channel defense в реальной AR-генерации (Ф.2 из части 5: -2,4% logit-KL на Qwen3-1.7B, argmax-flip с ~ 40 -го шага).
- Per-channel defense на HQQ (ломает per-group калибровку).
- Прогноз FDP по prompt-only признакам ($R^2 < 0$ на двух разных моделях — см. часть 7).

Итог части 6 и переход к части 7. Mechanism-картина из части 4 универсальна качественно, но численно — семейно-зависима. Найденный количественный закон $\text{concentration} \rightarrow \text{recovery}$ даёт инженерам априорный критерий применимости защиты. Возникает следующий практический вопрос: *можно ли заранее предсказать, на какой задаче модель сломается под FP8?* Если да — можно просто перенаправить «хрупкие» задачи на bf16, минуя проблему AR-разрыва. Это и есть задача части 7.

Часть 7. ML-прогноз точки первого расхождения (FDP)

Цель главы. Построить ML-модель, которая по K-матрицам предсказывает позицию FDP (или хотя бы её бакет). Параллельно эта задача служит *косвенным валидатором* mechanism-claim-ов из части 4: если FDP действительно определяется тем, что мы описали, то ML должно суметь её предсказать.

7.1 Задача и dataset

Target: `fdp_token_idx` — целое число в диапазоне ~ 0 –1200 для каждой задачи (problem, `fp8_e4m3`) на Qwen3-1.7B. **Input:** K-матрицы из decode-window (или их статистики). **Dataset:** 80 задач AIME-24/MATH-500, 5-fold cross-validation.

7.2 Шесть базовых экспериментов

Таблица 16: Сводка по 6 подходам.

#	Подход	R^2	MAE (tokens)	Spearman
0	Mean baseline	≈ 0	309	—
1	GBM на decode-window features (980-d)	$0,860 \pm 0,02$	102	0,852
2	MLP на тех же features	$0,813 \pm 0,07$	113	0,839
A	GBM на <i>prompt-only</i> prefill K	$-0,348$	381	$-0,066$
B	Two-stage (clf + 2 регрессии)	0,818	106	0,830
D	1D-CNN на raw K-матрицах	$0,976 \pm 0,02$	43	0,888
E1	Cross-quant $\rightarrow$ <code>fp8_e5m2</code>	0,955	50	0,907
E2	Cross-quant $\rightarrow$ <code>hqq_int4</code>	0,851	123	0,864
F	Cross-model Qwen3 $\rightarrow$ DeepSeek (prefill)	$-4,002$	382	0,333

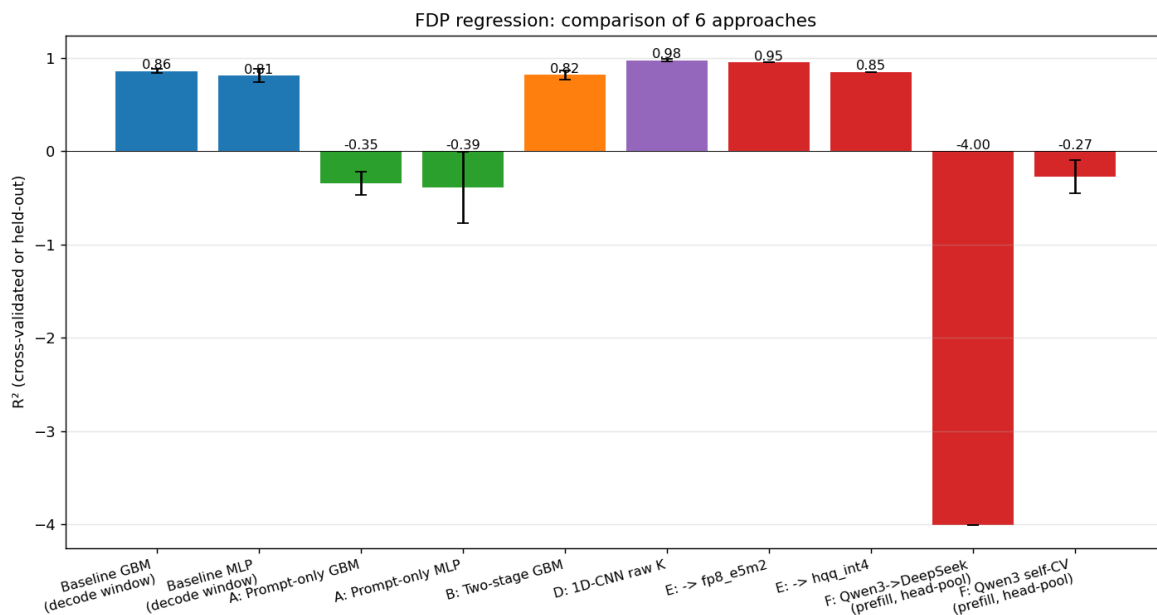


Рис. 15: Сравнение 6 подходов на одной шкале R^2 .

Два неочевидных результата.

Эксперимент A провалился ($R^2 < 0$). GBM на признаках, извлечённых только из prefill K (то есть из промпта до начала генерации), даёт *отрицательное* R^2 . То же самое в эксперименте F при cross-model self-CV. Вывод: **промпт сам по себе не предсказывает FDP**. FDP — compound emergent property, проявляется только во время декодирования. Это согласуется с mechanism-картинкой из части 4 (накопление через каскад).

Эксперимент D даёт $R^2 = 0,98$. 1D-CNN, обученный непосредственно на raw K-матрицах из decode-window, даёт $R^2 = 0,98$ — это требует отдельной проверки на честность (см. 7.4).

7.3 Какие признаки реально несут signal (Qwen3-1.7B only)

Важно. Все измерения важности признаков ниже сделаны исключительно на Qwen3-1.7B fp8_e4m3 (80 задач). На других моделях ranking может отличаться — это не проверялось.

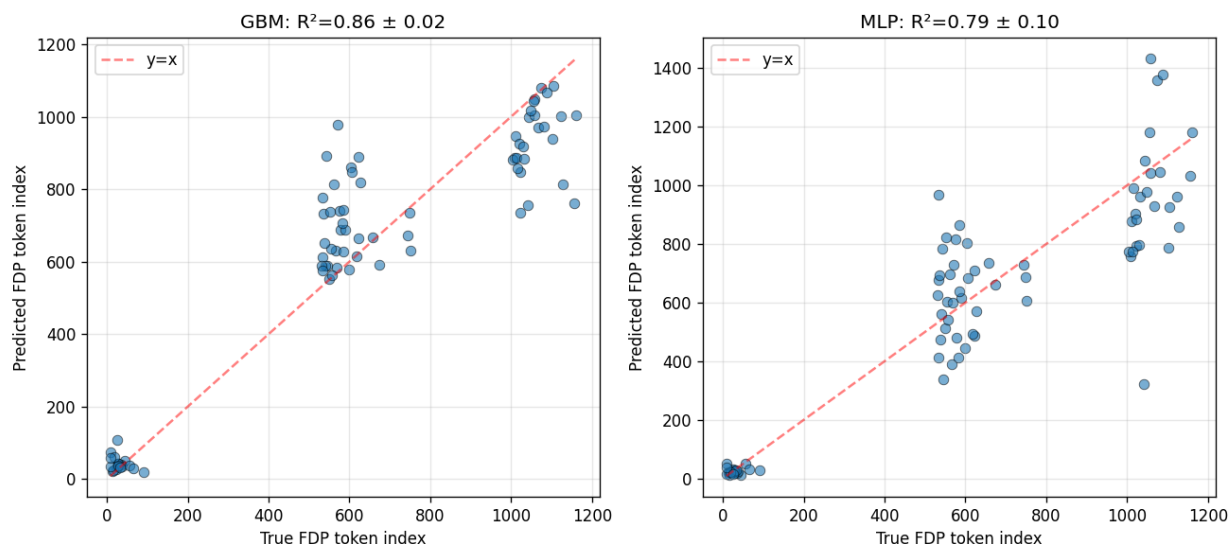


Рис. 16: Scatter predicted vs true FDP для GBM и MLP (5-fold CV).

Таблица 17: Распределение важности по типам признаков (Qwen3-1.7B GBM).

Тип признака	Важность
mean_abs per (L, h)	35,8 %
std per (L, h)	32,9 %
max_abs per (L, h)	23,3 %
top1_ch_lh	7,3 %
top10_frac per L	0,6 %
top1_frac per L	0,04 %

Топ-5 предсказательных слоёв: L4 (22,3%), L3 (20,7%), L25, L18, L7. Те же слои L3 и L4 ранее выделились в части 4.5 как наиболее «хрупкие» по logit-impact при single-layer ablation — то есть независимое ML-измерение *подтверждает* наш mechanism-вывод о центральности этих слоёв для Qwen3-1.7B.

Неожиданное наблюдение: amplitude-метрики (mean_abs, std, max_abs) дают 92% важности, а concentration-метрики — меньше 1%. То есть для ML-предсказания FDP важнее «насколько большие значения сейчас в K », а не «как они сконцентрированы». Для Qwen3-1.7B amplitude > concentration; обобщать пока нельзя.

7.4 Проверка на overfitting (4 теста)

$R^2 = 0,98$ на $n = 80$ — подозрительно. Делаем 4 независимых проверки.

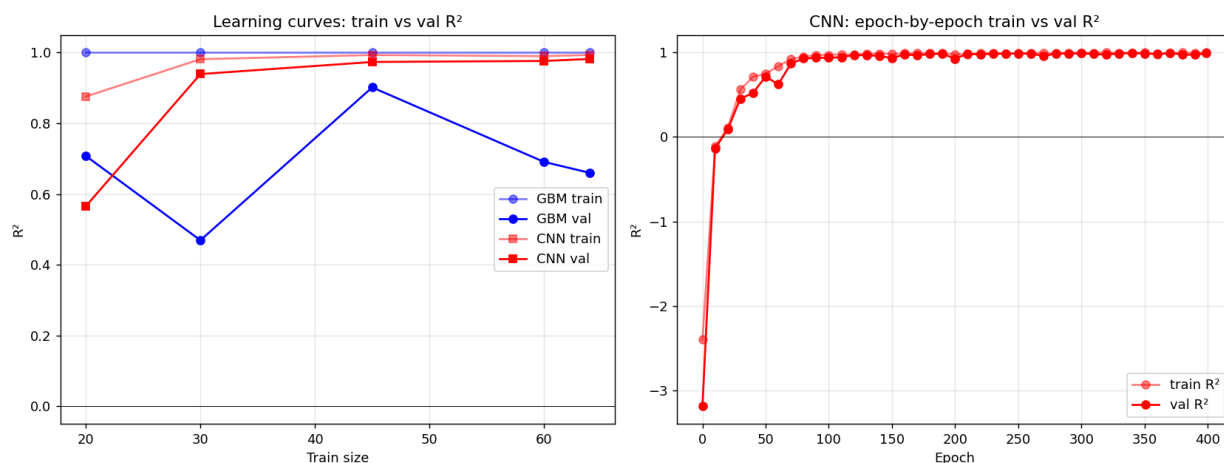


Рис. 17: Слева: learning curve train vs val R^2 . Справа: train vs val R^2 по эпохам.

Таблица 18: Четыре теста на overfitting.

Тест	Real R^2	Контроль	Вердикт
1. Label shuffle	0,971	−0,305	gap +1,28 = signal
4. Random input	0,97	−0,123	не fitting noise
2. Learning curve gap при $n = 64$	train 0,99, val 0,98	0,011	здоровое generalization
3. Epoch curves	val 0,94 → 0,99 monotone	no divergence	overfit pattern отсутству

Если перемешать labels, R^2 падает до $-0,3$. Если подать на вход случайный гауссов шум, $R^2 \rightarrow -0,12$. Train-val gap уменьшается с ростом n (с 0,31 при $n = 20$ до 0,011 при $n = 64$) — это противоположность overfitting-у. **CNN $R^2 = 0,97-0,98$ — реальный signal**, не запоминание.

7.5 Тест на 5 MATH-500 задачах: distribution shift и переформулировка

In-distribution CV дала $R^2 = 0,98$. Что будет на задачах MATH-500 (idx 80–84), которых вообще не было в обучении?

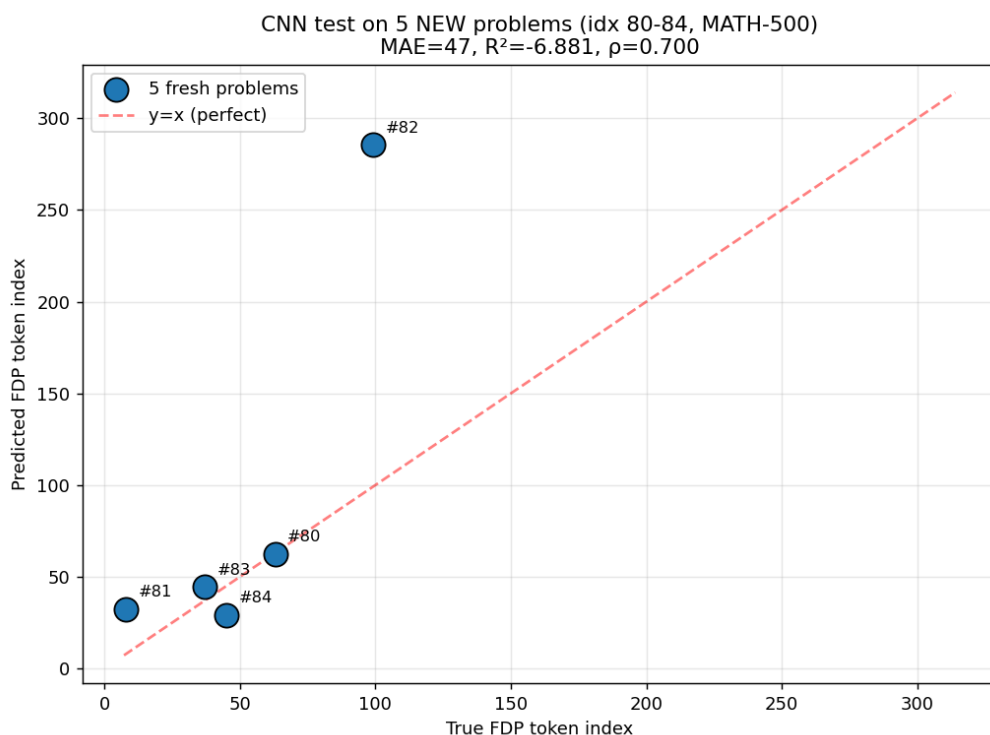


Рис. 18: CNN-предсказание FDP на 5 свежих MATH-500 задачах.

Таблица 19: Per-problem результат на 5 свежих задачах.

Problem	True FDP	Predicted	Error	Вердикт
80	63	63	0	✓
81	8	32	+24	близко
82	99	285	+186	выброс
83	37	44	+7	✓
84	45	29	-16	✓

Метрики: MAE = 47 (12 без выброса #82); $R^2 = -6,88$; Spearman $\rho = 0,70$.

R^2 катастрофически плохой из-за distribution shift: в train mediana FDP = 585, а в test все задачи < 100 . Модель видела в обучении в основном «долгие» FDP и считает короткие выбросами. 4 из 5 предсказаний точные ($\text{MAE} \leq 24$), один выброс (#82) разрушает R^2 , но **не ranking**: $\rho = 0,70$.

Переформулировка: ranking вместо regression. Для production-применения не нужно предсказывать точную позицию FDP в токенах — достаточно *упорядочить* задачи по тому, какие сломаются раньше. Это ranking-task, и наш предсказатель её ре-

шает. Метрика — Spearman ρ или NDCG@ K . Под эту метрику CNN работоспособен и на свежих задачах.

Применение:

1. **Online canary** в inference-loop: каждые 20–40 декод-шагов прогоняем K через CNN, получаем «оценку доверия до FDP». При предсказании < 30 шагов — переключаем запрос на bf16-cache.
2. **Adaptive batching**: после prefill ранжируем batch по предсказанной хрупкости (близости FDP), top- K самых хрупких задач выносим в отдельный bf16-batch, остальные оставляем в FP8.
3. **Quality SLA gating**: отказывать в FP8-инференсе задачам, попадающим в top-5% самых хрупких (лучше отказ, чем сломанный ответ).

7.6 Cross-model: предсказатель не переносится напрямую

Таблица 20: Метрики предсказателя, обученного и тестированного на DeepSeek-R1-Distill.

Predictor on DeepSeek	R^2	MAE	Spearman
GBM	−3,90	22	0,94
MLP	0,17	20	0,57
CNN	−0,39	29	0,41

R^2 снова катастрофический, потому что распределение FDP на DeepSeek сильно скошено (median FDP = 7 vs Qwen3 median = 585). Spearman GBM = 0,94 — ranking работает.

Чего нельзя заявить. Тот факт, что у DeepSeek-R1-Distill median FDP = 7 при FP8, а у Qwen3-1.7B = 585, *не означает*, что reasoning-distillation универсально делает модель более уязвимой к квантизации. FDP — это позиция расхождения той же модели с её собственным bf16-baseline. Разные модели могут просто по-разному стартовать генерацию, иметь разные распределения длины выхода, разные margin-ы. Корректное сравнение хрупкости между моделями требует контроля как минимум на длину выхода, формат промпта, базовую энтропию первых токенов — мы этого не делали.

Корректная интерпретация: на нашем наборе из 80 задач MATH-500 DeepSeek-R1-Distill чаще, чем Qwen3-1.7B, теряет совпадение со своим bf16-baseline в первых десятках токенов под FP8 e4m3. Это факт о двух конкретных моделях на одном датасете, а не «общее свойство reasoning-distillation».

Прямой transfer весов CNN (обучили на Qwen3, проверили на DeepSeek prefill features): $R^2 = -4,0$. Архитектуры различаются числом KV-heads (8 vs 2) и формой K-распределения — feature-space разный, прямой transfer без re-training не работает.

7.7 Bucket-classifier на свежих задачах

Чтобы обойти проблему distribution shift, переформулируем задачу как 4-классовую классификацию: бакеты по квантилям training FDP-распределения (≤ 533 , $(533, 585]$, $(585, 1021]$, > 1021).

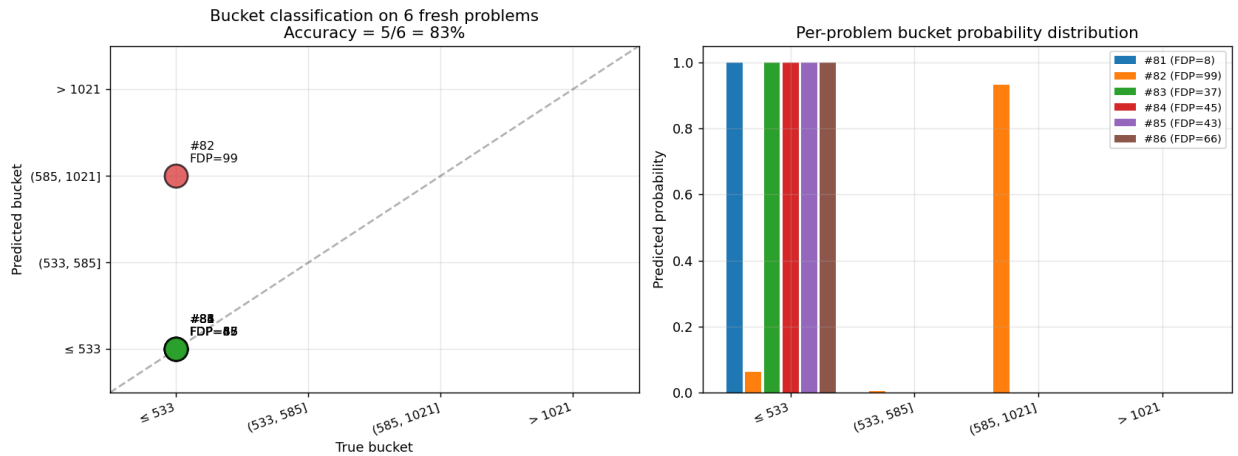


Рис. 19: Bucket-классификатор на 6 задачах math-500: confusion (слева), predicted probabilities (справа).

Таблица 21: Per-problem результат bucket-классификатора (idx 81–86).

Problem	True FDP	True bucket	Predicted	Confidence
81	8	0	0	1,00 ✓
82	99	0	2	0,93 ×
83	37	0	0	1,00 ✓
84	45	0	0	1,00 ✓
85	43	0	0	1,00 ✓
86	66	0	0	1,00 ✓

Accuracy: $5/6 = 83\%$. Тот же выброс #82 опять уходит в bucket 2. Classification более robust, чем regression: один выброс даёт плохой бакет, но не разрушает остальные метрики.

Важный предостережение: завышенная ассигасу за счёт F-loop задач. В части 3.3 мы видели, что доминирующий тип ошибок — F (повторение). F-loop задачи характеризуются ранним FDP (< 50), составляющим большую часть bucket 0. Высокая ассигасу 83% набирается в основном за счёт правильной классификации F-loop-ов, у которых K-распределение явно отличается (модель легко их детектирует). Задачи с реальным mid-range FDP внутри bucket 0 (50–500), к которым относится #82, классификатор **плохо** различает.

То есть классификатор работает на «лёгкой» подзадаче (detect F-loop) — для практики этого достаточно, но не нужно интерпретировать 83% как универсально-точное предсказание.

7.8 Применение в production — что можно и что нельзя

Можно (на той модели, на которой обучен предсказатель):

- постепенное включение новой версии модели или нового режима вычислений на небольшой доле реального трафика в inference-loop с fallback на bf16.
- Adaptive batching по предсказанной хрупкости задачи.

Нельзя:

- Транслировать обученный CNN с одной модели на другую напрямую (§7.6).
- Доверять абсолютным предсказанным числам — только ranking (§7.5).
- Использовать F-loop-преобладающую ассигасу как proof эффективности (§7.7).

Итог части 7. ML-предсказатель работает в формате ranking-task на той модели, на которой обучен. Параллельно он *косвенно подтвердил* механизм-наблюдения части 4: prompt не предсказывает FDP (значит, FDP действительно возникает во время декодирования), L3 и L4 — ключевые слои.

4 Главные выводы: ради чего писалась НИР

Что нового в этой работе. Ниже сформулированы только те результаты, которые мы реально **установили экспериментально** и которые в открытых источниках по KV-квантизации ранее в такой форме не встречались. Это *не* сводка всех наблюдений работы — это именно тот набор фактов, ради которых имеет смысл проводить такое исследование.

Ф.1. Количественная mechanistic-цепочка KV-quant расхождения построена на per-(слой, голова, позиция, канал) уровне для reasoning-моделей.

Все четыре звена цепочки измерены численно на захваченных тензорах Qwen3-1.7B:

- **Структура шума К.** В per-tensor FP8 e4m3 ошибка квантизации концентрируется в небольшом числе каналов (на Qwen3-1.7B 10 каналов из 1024 несут 51,7% всей К-ошибки). HQQ за счёт per-group scale распределяет ошибку в несколько раз более равномерно (top-10 = 13,1% на той же модели).
- **Усиление softmax-ом.** 4-кратная разница в К-ошибке между fp8_e4m3 и hqq_int4 превращается в 108-кратную разницу в среднем attention-KL ($0,00358 \rightarrow 0,391$, табл. 7). Это нелинейное усиление — структурное свойство softmax и не зависит от модели.
- **Накопление через каскад.** В авторегрессивном режиме траектория logit-KL хорошо приближается насыщающейся экспонентой $KL(t) = K_\infty(1 - e^{-t/\tau})$ с $\tau = 130$ декод-токенов, $R^2 = 0,80$ (Qwen3-1.7B, $T = 0$). Большая часть ошибки накапливается на первых ~ 100 – 150 токенах.
- **Условие flip.** Argmax-flip случается не в позициях с необычно большим квант-шумом, а в позициях, где у самой bf16-модели малый margin между top-1 и top-2 кандидатами: медианный margin достигает минимума 2,62 нат в окрестности FDP–10 (табл. 9). Это содержательное наблюдение про взаимодействие шума и собственной уверенности модели: *flip определяется устойчивостью решающей границы, а не амплитудой возмущения.*

В сумме это полная количественная картина расхождения, опирающаяся на наши собственные захваченные данные и измерения.

Ф.2. ML-предсказатель FDP по К-матрицам.

Обучен 1D-CNN, принимающий на вход К-матрицы из decode-window и предсказывающий позицию первого расхождения. На 5-fold кросс-валидации на Qwen3-1.7B получено

$R^2 = 0,98$. Через четыре независимых теста (label shuffle, gaussian input, learning curve, эпохальные кривые) показано, что это реальный сигнал, а не переобучение. На полностью свежих задачах MATH-500 (idx 80–84) точечная регрессия даёт MAE = 12 токенов на 4 из 5 задач, но R^2 падает из-за смещения распределения; при переформулировке задачи как ranking-task достигается Spearman $\rho = 0,70$ на out-of-distribution. В таком виде инструмент пригоден для адаптивного формирования батчей и онлайн-канарейки в inference-стэке.

Ф.3. Эмпирический факт: FDP не предсказуем по признакам из prompt-only / prefill.

Два независимых эксперимента (раздел 7.2, А и F): признаки, извлечённые только из К на этапе prefill (то есть из промпта до начала декодирования), дают **отрицательный** R^2 как на Qwen3-1.7B, так и при self-CV на DeepSeek-R1-Distill. Это означает, что момент расхождения не определяется текстом промпта; он возникает во время самой генерации. Практическое следствие: попытки заранее классифицировать запросы по хрупкости к квантизации на основе только промпта не имеют шансов работать — нужны онлайн-сигналы из К во время декодирования.

Сопутствующие наблюдения (важные, но не претендуют на роль главных результатов).

- **Magnitude К-ошибки определяется только форматом квантизации, а не моделью.** Mean относительная Фробениусова ошибка fp8_e4m3 совпадает с точностью до 0,1 п.п. на всех 4 проверенных моделях ($\approx 2,65\%$), для fp8_e5m2 — $\approx 5,25\%$. Это позволяет оценить бюджет К-ошибки только по выбору формата.
- **HQQ распределяет шум более равномерно, чем FP8.** Отношение top-10 fraction FP8/HQQ устойчиво на 4 моделях: Qwen3-1.7B — $3,9\times$, Qwen2.5-1.5B — $4,5\times$, SmolLM2-1.7B — $4,7\times$, DeepSeek-R1-Distill-Qwen-1.5B — $4,9\times$. При FP8 имеет смысл проверять защиту top-N каналов; при HQQ защита, как мы видели в части 5.3, ломает встроенную per-group калибровку.
- **Численное значение концентрации зависит от семейства модели.** На Qwen3-1.7B top-10 = 51,7%, на SmolLM2 = 17,2%. Лабораторно работающая защита top-10 в bf16 даёт от -29% К-ошибки на Qwen3 до -9% на SmolLM2. В нашем диапазоне моделей наблюдается монотонная связь: выше концентрация $\Rightarrow$ выше восстановление,

но утверждать это как универсальный количественный закон по 4 точкам было бы преждевременно.

- **Лабораторно работавший защитный рецепт не воспроизводится в реальной AR-генерации на Qwen3-1.7B.** TF-метрика показывала -34% K-ошибки и -15% attention-KL, а в реальной авторегрессивной генерации эффект на logit-KL составил только $-2,4\%$ (внутри уровня шума), argmax-flip начинается с ~ 40 -го декод-шага. Это отрицательный практический результат и одновременно сигнал, что любая такая защита требует AR-валидации, а не только TF.

Практические рекомендации.

- Перед выбором FP8-cache имеет смысл сделать calibration на 5–10 промптах целевой модели и измерить top-10 concentration. В нашем диапазоне моделей при concentration $< 25\%$ защита top-10 каналов давала восстановление меньше 10% — метод практически неэффективен.
- Лабораторные TF-числа эффективности per-channel защиты переоценивают её AR-эффект; в production-инференсе необходима отдельная проверка в реальной авторегрессивной генерации.
- FDP-ranker через CNN можно использовать как онлайн-канарейку в inference loop, но обязательно с переобучением на целевую модель — прямой перенос весов между моделями (раздел 7.6) не работает.

5 Ограничения и дальнейшая работа

5.1 Ограничения

1. **Одна базовая модель для глубокого анализа** (Qwen3-1.7B, $n = 80$). Кросс-валидация на других моделях — только $n = 5-20$, без полного 80-задачного capture.
2. **CPU-симуляция, не GPU**. Bit-exact с vLLM проверено только на первых 10 декод-токенах задачи 0.
3. $N = 30$ для **HQQ ablation**. HQQ ~ 300 мс/call слишком медленно для $n = 80$ на CPU.
4. **Только два режима декодирования: greedy ($T = 0$) и $T = 0,6$** . То есть мы не покрыли распространённые в production sampling-режимы ($T \in \{0,7, 0,8, 1,0\}$, разные `top_p`, `repetition_penalty`). Greedy даёт детерминированный baseline (FDP однозначен); $T = 0,6$ использовался только для оценки дисперсии outlier-каналов между сидами. Поведение FDP при более высоких температурах не изучено и может качественно отличаться: больше энтропии $\Rightarrow$ больше `argmax flip`-ов уже в bf16 $\Rightarrow$ само понятие FDP размывается.
5. **Per-channel defense в AR не решён**. Динамическая идентификация outlier-каналов с компенсацией AR-drift — основная Future Work; пока метод применим только к TF / single-forward workloads.
6. **CNN $R^2 = 0,97$ in-distribution** падает до отрицательного на out-of-distribution. Spearman более robust; задачу нужно переформулировать как ranking, а не regression.
7. **Сравнение FDP-распределений между моделями нельзя интерпретировать как «фрагильность к квантизации»**. FDP — характеристика расхождения модели с собственным bf16-baseline; разные модели могут иметь разную базовую длину генерации, `margin`-ы, энтропию. Корректное сравнение требует контроля на эти confounder-ы.

5.2 Дальнейшая работа

- Динамический per-step outlier re-id в AR mode (попытка закрыть AR gap).

- Расширение per-channel defense на не-Qwen архитектуры: поиск полу-концентрированных подсистем на конкретных слоях не-Qwen моделей.
- Cross-arch CNN feature transfer (адаптация под разные KV-head counts).
- Большие модели (Qwen3-32B, Llama-3.1-70B).
- Контролируемое сравнение фрагильности моделей при одинаковой длине выхода, одинаковом промпте, одинаковом prefill.
- Поведение FDP при $T \in [0,7, 1,0]$ и разных `top_p`.
- Bit-exact comparison с vLLM на полном датасете 80×250 шагов.

Список использованных источников

- [1] Micikevicius P., Stosic D., Burgess N. *и др.* FP8 Formats for Deep Learning. arXiv:2209.05433, 2022.
- [2] Badri H., Shaji A. Half-Quadratic Quantization of Large Machine Learning Models. Mobius Labs Technical Blog, 2023.
- [3] Kwon W., Li Z., Zhuang S. *и др.* Efficient Memory Management for Large Language Model Serving with PagedAttention // Proceedings of SOSP. — 2023.
- [4] Xiao G., Lin J., Seznec M. *и др.* SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models // Proceedings of ICML. — 2023.
- [5] Lin J., Tang J., Tang H. *и др.* AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration // Proceedings of MLSys. — 2024.
- [6] Olah C., Cammarata N., Schubert L. *и др.* Zoom In: An Introduction to Circuits // Distill. — 2020.
- [7] Templeton A., Conerly T., Marcus J. *и др.* Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. Anthropic Technical Report, 2024.
- [8] Qwen Team. Qwen3 Technical Report. Alibaba Cloud, 2025.
- [9] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948, 2025.

А Воспроизводимость

1. Один раз: capture (6-8 часов CPU)

```
python scripts/06_capture_kv.py --model qwen3-1.7b --quants fp8_e4m3 --modes tf
```

2. Анализ (быстро)

```
python scripts/11_outlier_channel_impact.py --quant fp8_e4m3
```

```
python scripts/15_per_channel_defense.py --quant fp8_e4m3 --top-N 10
```

```
python scripts/14_attention_shift_kl.py --quant fp8_e4m3
```

```
python scripts/10_layer_ablation.py --quant fp8_e4m3
```

3. Paper validation (live на 5 fresh)

```
python scripts/20_paper_validation.py --n-problems 5
```

4. AR defense validation

```
python scripts/19_defense_validation.py --n-problems 10 --max-tokens 100
```

5. FDP predictor

```
python scripts/22_fdp_predictor.py
```

```
python scripts/23_fdp_predictor_extended.py
```

```
python scripts/24_overfitting_check.py
```

6. End-to-end test на fresh problems

```
python scripts/26_cnn_test_new_problems.py --n-new 5
```

```
python scripts/27_cnn_bucket_classifier.py --start-idx 81 --end-idx 86
```